

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH

BÁO CÁO LAB 1

Kafka & Spark

Môn học: Dữ liệu lớn (CO3138)

Học kì: HK252

Giảng viên: Đoàn Ngô Đức Phương

STT	Họ và tên	MSSV
1	Nguyễn Chánh Tín	2213491
2	Trần Quốc Toàn	2213540

Mục lục

1. Tổng quan	3
2. Môi trường thực hiện	3
3. Triển khai cụm Kafka	3
4. Khởi tạo Spark Session	4
5. Chuẩn bị dữ liệu	4
5.1. Tải tập dữ liệu MovieLens	4
5.2. Tạo Kafka topics và nạp dữ liệu	5
6. Exercise	6
6.1. Exercise 1 — Đọc dữ liệu từ Kafka và xử lý lỗi định dạng	6
6.1.1. Vấn đề	6
6.1.2. Giải pháp	6
6.2. Exercise 2 — Top 5 phim có điểm đánh giá trung bình cao nhất	8
6.2.1. Phương pháp	8
6.2.2. Kết quả	8
6.3. Exercise 3 — 5 tag tệ nhất (gắn với điểm đánh giá trung bình thấp nhất)	8
6.3.1. Phương pháp	8
6.3.2. Kết quả	9
6.4. Exercise 4 — Phân tích sâu: Các tag tệ có thực sự phản ánh chất lượng phim?	9
6.4.1. Đặt vấn đề	9
6.4.2. Phương pháp	10
6.4.3. Kết quả chi tiết	11
6.4.4. Bảng tổng hợp	12
6.4.5. Nhận xét	12
7. Kết luận	13

1. Tổng quan

Bài thí nghiệm Lab 01 nhằm xây dựng kiến thức thực hành về hai nền tảng xử lý dữ liệu phân tán phổ biến: **Apache Spark** và **Apache Kafka**. Cụ thể, bài lab yêu cầu:

1. Triển khai cụm Kafka gồm 3 broker trên môi trường local bằng Docker.
2. Đọc dữ liệu từ Kafka, xử lý lỗi định dạng binary.
3. Phân tích tập dữ liệu MovieLens bằng Spark DataFrame:
 - Tìm top 5 phim có điểm đánh giá trung bình cao nhất (với số lượng rating > 30).
 - Tìm 5 tag tệ nhất (gắn với điểm đánh giá trung bình thấp nhất).
 - Phân tích sâu: các tag đó có thực sự phản ánh chất lượng phim thấp hay không.

2. Môi trường thực hiện

Thành phần	Chi tiết
Hệ điều hành	Windows 11
Python	3.13
PySpark	4.0.0
Java	OpenJDK 17 (Eclipse Temurin)
Docker	Docker Desktop for Windows
Kafka Image	apache/kafka:3.9.0
IDE	VS Code + Jupyter Lab

3. Triển khai cụm Kafka

Cụm Kafka được triển khai bằng Docker Compose với 3 broker sử dụng chế độ KRaft (không cần Zookeeper). Mỗi broker đóng vai trò đồng thời là controller và broker, giao tiếp qua 3 listener: PLAINTEXT (client), CONTROLLER (quorum), và INTERNAL (inter-broker).

Các broker được expose ra các port khác nhau trên máy host:

- Broker 1: localhost:9092
- Broker 2: localhost:9192
- Broker 3: localhost:9292

Lệnh triển khai:

```
docker compose --profile kafka up -d
```

```
C:\Users\ASUS>docker ps -n 3
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
46440f6b54c8	apache/kafka:3.9.0	"/__cacert_entrypoint..."	47 minutes ago	Up 47 minutes	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp	lab01-broker-1-1
94d116c34ff8	apache/kafka:3.9.0	"/__cacert_entrypoint..."	47 minutes ago	Up 47 minutes	0.0.0.0:9192->9092/tcp, [::]:9192->9092/tcp	lab01-broker-2-1
692725b80882	apache/kafka:3.9.0	"/__cacert_entrypoint..."	47 minutes ago	Up 47 minutes	0.0.0.0:9292->9092/tcp, [::]:9292->9092/tcp	lab01-broker-3-1

Hình 1: Cụm Kafka 3 broker đang hoạt động trên Docker.

4. Khởi tạo Spark Session

SparkSession được khởi tạo ở chế độ `local[*]` (sử dụng tất cả CPU cores), kèm theo các Kafka connector JARs cần thiết được khai báo qua `spark.jars.packages`.

```
23
24 spark.sparkContext.setLogLevel("WARN")
25 print(f"Spark version : {spark.version}")
26 print(f"Spark UI      : {spark.sparkContext.uiWebUrl}")
```

```
Spark version : 4.0.0
Spark UI      : http://host.docker.internal:4040
```

Hình 2: Khởi tạo SparkSession thành công — Spark 4.0.0.

5. Chuẩn bị dữ liệu

5.1. Tải tập dữ liệu MovieLens

Tập dữ liệu **MovieLens Latest Small** được tải về từ Kaggle thông qua thư viện `kagglehub`. Tập dữ liệu gồm 3 bảng chính:

- **ratings**: Thông tin đánh giá phim (`userId`, `movieId`, `rating`, `timestamp`).
- **movies**: Thông tin phim (`movieId`, `title`, `genres`).
- **tags**: Tag do người dùng gán cho phim (`userId`, `movieId`, `tag`, `timestamp`).

```

Dataset path: C:\Users\ASUS\.cache\kagglehub\datasets\grouplens\movielens-latest-small\versions\2
=== Ratings ===
+-----+-----+-----+-----+
|userId|movieId|rating|timestamp|
+-----+-----+-----+-----+
|1      |1      |4.0   |964982703|
|1      |3      |4.0   |964981247|
|1      |6      |4.0   |964982224|
+-----+-----+-----+-----+
only showing top 3 rows
=== Movies ===
+-----+-----+-----+-----+
|movieId|title                |genres
+-----+-----+-----+-----+
|1      |Toy Story (1995)     |Adventure|Animation|Children|Comedy|Fantasy|
|2      |Jumanji (1995)       |Adventure|Children|Fantasy
|3      |Grumpier Old Men (1995)|Comedy|Romance
+-----+-----+-----+-----+
only showing top 3 rows
=== Tags ===
+-----+-----+-----+-----+
|userId|movieId|tag          |timestamp |
+-----+-----+-----+-----+
|2      |60756  |funny        |1445714994|
|2      |60756  |Highly quotable|1445714996|
|2      |60756  |will ferrell  |1445714992|
+-----+-----+-----+-----+
only showing top 3 rows

```

Hình 3: Dữ liệu mẫu từ 3 bảng ratings, movies, tags.

5.2. Tạo Kafka topics và nạp dữ liệu

Ba topic (ratings, movies, tags) được tạo trên cụm Kafka với replication_factor=3 (mỗi broker giữ một bản sao). Sau đó, dữ liệu được serialize thành JSON và đẩy vào các topic tương ứng thông qua Spark Kafka connector.

```

13
14 # Create topics with replication factor = 3 (one replica per broker)
15 new_topics = [
16     NewTopic(topic="ratings", num_partitions=1, replication_factor=3),
17     NewTopic(topic="movies", num_partitions=1, replication_factor=3),
18     NewTopic(topic="tags", num_partitions=1, replication_factor=3),
19 ]
20 fs = admin_client.create_topics(new_topics)
21 for topic, f in fs.items():
22     try:
23         f.result()
24         print(f"Topic '{topic}' created successfully.")
25     except Exception as e:
26         print(f"Failed to create topic '{topic}': {e}")

```

```

Topic 'ratings' created successfully.
Topic 'movies' created successfully.
Topic 'tags' created successfully.

```

Hình 4: Tạo 3 Kafka topics thành công.

```

1 # ----- Push DataFrames to Kafka -----
2 for name, df_src in [("ratings", df_ratings), ("movies", df_movies), ("tags", df_tags)]:
3     df_src.selectExpr("to_json(struct(*) AS value)" \
4         .write \
5         .format("kafka") \
6         .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
7         .option("topic", name) \
8         .save()
9     print(f"Topic '{name}': {df_src.count()} records pushed.")
10
11 print("\nAll data successfully loaded into Kafka.")

```

Topic 'ratings': 100836 records pushed.
 Topic 'movies': 9742 records pushed.
 Topic 'tags': 3683 records pushed.

All data successfully loaded into Kafka.

Hình 5: Đẩy dữ liệu vào Kafka — hiển thị số lượng records mỗi topic.

6. Exercise

6.1. Exercise 1 — Đọc dữ liệu từ Kafka và xử lý lỗi định dạng

6.1.1. Vấn đề

Khi đọc dữ liệu từ Kafka, cột `value` được trả về dưới dạng **binary** (bytes). Nếu không xử lý, dữ liệu sẽ hiển thị dưới dạng mảng byte không đọc được.

```

[5]: # ----- Step 1: Read raw data from Kafka topics -----
def read_kafka_topic(topic: str) -> "DataFrame":
    """Read a Kafka topic and return the raw DataFrame."""
    return (
        spark.read
        .format("kafka")
        .option("kafka.bootstrap.servers", KAFKA_BROKERS)
        .option("subscribe", topic)
        .option("startingOffsets", "earliest")
        .load()
    )

raw_ratings = read_kafka_topic("ratings")
raw_movies = read_kafka_topic("movies")
raw_tags = read_kafka_topic("tags")

print("Raw Kafka schema (ratings):")
raw_ratings.printSchema()
print("Raw value sample (binary format - this is the 'error'):")
raw_ratings.select("value").show(3, truncate=False)

```

Raw Kafka schema (ratings):

```

root
 |-- key: binary (nullable = true)
 |-- value: binary (nullable = true)
 |-- topic: string (nullable = true)
 |-- partition: integer (nullable = true)
 |-- offset: long (nullable = true)
 |-- timestamp: timestamp (nullable = true)
 |-- timestampType: integer (nullable = true)

```

Raw value sample (binary format - this is the 'error'):

```

+-----+
|value|
+-----+
[[78 22 75 73 65 72 49 64 22 3A 31 2C 22 6D 6F 76 69 65 49 64 22 3A 31 2C 22 72 61 74 69 6E 67 22 3A 34 2E 30 2C 22 74 69 6D 65 73 74 61 6D 70 22 3A 39 36 34 39 38 32 37 30 33 7D]]
[[78 22 75 73 65 72 49 64 22 3A 31 2C 22 6D 6F 76 69 65 49 64 22 3A 33 2C 22 72 61 74 69 6E 67 22 3A 34 2E 30 2C 22 74 69 6D 65 73 74 61 6D 70 22 3A 39 36 34 39 38 31 32 34 37 7D]]
[[78 22 75 73 65 72 49 64 22 3A 31 2C 22 6D 6F 76 69 65 49 64 22 3A 36 2C 22 72 61 74 69 6E 67 22 3A 34 2E 30 2C 22 74 69 6D 65 73 74 61 6D 70 22 3A 39 36 34 39 38 32 32 32 34 7D]]
+-----+

```

only showing top 3 rows

Hình 6: Dữ liệu thô từ Kafka — cột `value` ở dạng binary (lỗi định dạng).

6.1.2. Giải pháp

Pipeline xử lý gồm 3 bước:

Bước	Thao tác	Mục đích
1	CAST(value AS STRING)	Chuyển binary → chuỗi JSON (UTF-8)
2	from_json(col, schema)	Parse chuỗi JSON thành struct có kiểu dữ liệu
3	select("data.*")	Trải phẳng struct thành các cột riêng biệt

Schema được định nghĩa tường minh cho từng topic để đảm bảo kiểu dữ liệu chính xác (IntegerType, DoubleType, StringType, LongType).

```
=== Parsed Ratings (from Kafka) ===
+-----+-----+-----+-----+
|userId|movieId|rating|timestamp|
+-----+-----+-----+-----+
|1      |1       |4.0   |964982703|
|1      |3       |4.0   |964981247|
|1      |6       |4.0   |964982224|
|1      |47      |5.0   |964983815|
|1      |50      |5.0   |964982931|
+-----+-----+-----+-----+
only showing top 5 rows
root
|-- userId: integer (nullable = true)
|-- movieId: integer (nullable = true)
|-- rating: double (nullable = true)
|-- timestamp: long (nullable = true)

=== Parsed Movies (from Kafka) ===
+-----+-----+-----+-----+
|movieId|title                                     |genres
+-----+-----+-----+-----+
|1      |Toy Story (1995)                         |Adventure|Animation|Children|Comedy|Fantasy|
|2      |Jumanji (1995)                           |Adventure|Children|Fantasy
|3      |Grumpier Old Men (1995)                  |Comedy|Romance
|4      |Waiting to Exhale (1995)                 |Comedy|Drama|Romance
|5      |Father of the Bride Part II (1995)       |Comedy
+-----+-----+-----+-----+
only showing top 5 rows
=== Parsed Tags (from Kafka) ===
+-----+-----+-----+-----+
|userId|movieId|tag          |timestamp |
+-----+-----+-----+-----+
|2      |60756  |funny        |1445714994|
|2      |60756  |Highly quotable|1445714996|
|2      |60756  |will ferrell  |1445714992|
|2      |89774  |Boxing story  |1445715207|
|2      |89774  |MMA           |1445715200|
+-----+-----+-----+-----+
only showing top 5 rows
Ratings count : 100836
Movies count   : 9742
Tags count     : 3683
```

Hình 7: Dữ liệu sau khi parse — schema đúng, dữ liệu đọc được.

6.2. Exercise 2 — Top 5 phim có điểm đánh giá trung bình cao nhất

6.2.1. Phương pháp

1. Nhóm bảng ratings theo movieId, tính avg(rating) và count(rating).
2. Lọc các phim có rating_count > 30 để đảm bảo tính đại diện thống kê.
3. Join với bảng movies để lấy tên phim.
4. Sắp xếp giảm dần theo avg_rating, lấy top 5.

6.2.2. Kết quả

4. Exercise 2 — Top 5 Movies by Average Rating (rating count > 30)

```
# Aggregate ratings per movie
movie_stats = (
    ratings
    .groupBy("movieId")
    .agg(
        _round(avg("rating"), 2).alias("avg_rating"),
        count("rating").alias("rating_count")
    )
    .filter(col("rating_count") > 30)
)

# Join with movie titles and sort
top5_movies = (
    movie_stats
    .join(movies, on="movieId", how="inner")
    .select("movieId", "title", "genres", "avg_rating", "rating_count")
    .orderBy(desc("avg_rating"), desc("rating_count"))
    .limit(5)
)

print("=" * 80)
print("TOP 5 MOVIES BY AVERAGE RATING (rating count > 30)")
print("=" * 80)
top5_movies.show(truncate=False)
```

```
=====
TOP 5 MOVIES BY AVERAGE RATING (rating count > 30)
=====
```

movieId	title	genres	avg_rating	rating_count
318	Shawshank Redemption, The (1994)	Crime Drama	4.43	317
1204	Lawrence of Arabia (1962)	Adventure Drama War	4.3	45
858	Godfather, The (1972)	Crime Drama	4.29	192
2959	Fight Club (1999)	Action Crime Drama Thriller	4.27	218
750	Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)	Comedy War	4.27	97

Hình 8: Top 5 phim có điểm đánh giá trung bình cao nhất (rating count > 30).

Điều kiện rating_count > 30 giúp loại bỏ các phim có quá ít đánh giá — những phim này dễ có điểm trung bình cực đoan (rất cao hoặc rất thấp) nhưng không mang ý nghĩa thống kê.

6.3. Exercise 3 — 5 tag tệ nhất (gắn với điểm đánh giá trung bình thấp nhất)

6.3.1. Phương pháp

1. Join bảng tags với ratings theo movieId để gán rating cho mỗi tag.
2. Chuẩn hóa text tag (lowercase + trim) để gộp các biến thể giống nhau.
3. Nhóm theo tag_clean, tính avg(rating).

4. Sắp xếp tăng dần, lấy 5 tag có avg_rating thấp nhất.

6.3.2. Kết quả

5. Exercise 3 — 5 Worst Tags (Lowest Average Rating)

```
# Join tags with ratings on movieId to get the rating associated with each tag
tags_with_ratings = tags.join(ratings, on="movieId", how="inner")

# Normalize tag text to lowercase for consistent grouping
from pyspark.sql.functions import lower, trim

worst_tags = (
    tags_with_ratings
    .withColumn("tag_clean", lower(trim(col("tag"))))
    .groupBy("tag_clean")
    .agg(
        _round(avg("rating"), 2).alias("avg_rating"),
        count("rating").alias("rating_count")
    )
    .orderBy(asc("avg_rating"))
    .limit(5)
)

print("=" * 80)
print("5 WORST TAGS (associated with lowest average rating)")
print("=" * 80)
worst_tags.show(truncate=False)
```

```
=====
5 WORST TAGS (associated with lowest average rating)
=====
+-----+-----+-----+
|tag_clean|avg_rating|rating_count|
+-----+-----+-----+
|symbolic |0.5      |1          |
|stage    |1.75     |2          |
|tokyo    |2.0      |1          |
|snl      |2.1      |10         |
|jungle   |2.14     |11         |
+-----+-----+-----+
```

Hình 9: 5 tag có điểm đánh giá trung bình thấp nhất.

6.4. Exercise 4 — Phân tích sâu: Các tag tệ có thực sự phản ánh chất lượng phim?

6.4.1. Đặt vấn đề

Kết quả Exercise 3 cho thấy một số tag gắn liền với điểm trung bình thấp. Tuy nhiên, câu hỏi đặt ra: liệu các phim mang những tag đó có thực sự bị đánh giá thấp, hay chỉ là ảnh hưởng của một vài đánh giá cá biệt?

6.4.2. Phương pháp

1. Với mỗi tag trong 5 tag tệ nhất, tìm tất cả các phim mang tag đó.
2. Tính **điểm đánh giá trung bình riêng** của từng phim (từ toàn bộ ratings, không chỉ từ người gắn tag).
3. Đếm số phim mang mỗi tag.
4. So sánh mean_of_movie_avgs (trung bình của các trung bình phim) với **điểm trung bình toàn cục** (global average).

6.4.3. Kết quả chi tiết

```
# For each worst tag: find associated movies, their individual avg ratings, and movie count
from pyspark.sql.functions import countDistinct

# Prepare: tags with cleaned text joined with ratings
tags_clean = tags.withColumn("tag_clean", lower(trim(col("tag"))))

# Filter only movies that have any of the 5 worst tags
worst_tag_movies = (
    tags_clean
    .filter(col("tag_clean").isin(worst_tag_names))
    .select("tag_clean", "movieId")
    .distinct()
)

# Join with ratings to compute per-movie average rating for these tagged movies
worst_tag_movie_ratings = (
    worst_tag_movies
    .join(ratings, on="movieId", how="inner")
    .groupBy("tag_clean", "movieId")
    .agg(_round(avg("rating"), 2).alias("movie_avg_rating"))
)

# Join with movie titles for readability
worst_tag_movie_detail = (
    worst_tag_movie_ratings
    .join(movies, on="movieId", how="inner")
    .select("tag_clean", "movieId", "title", "movie_avg_rating")
    .orderBy("tag_clean", asc("movie_avg_rating"))
)

print("=" * 80)
print("PER-MOVIE AVERAGE RATINGS FOR MOVIES WITH THE 5 WORST TAGS")
print("=" * 80)
worst_tag_movie_detail.show(50, truncate=False)
```

```
=====
PER-MOVIE AVERAGE RATINGS FOR MOVIES WITH THE 5 WORST TAGS
=====
+-----+-----+-----+-----+
|tag_clean|movieId|title|movie_avg_rating|
+-----+-----+-----+-----+
|jungle   |1474   |Jungle2Jungle (a.k.a. Jungle 2 Jungle) (1997)|2.14|
|snl      |2296   |Night at the Roxbury, A (1998)|2.1|
|stage    |8943   |Being Julia (2004)|1.75|
|symbolic |26717  |Begotten (1990)|0.5|
|tokyo    |6407   |Walk, Don't Run (1966)|2.0|
+-----+-----+-----+-----+
```

Hình 10: Điểm đánh giá trung bình của từng phim mang các tag tệ nhất.

6.4.4. Bảng tổng hợp

```
# Summary statistics per tag
tag_summary = (
    worst_tag_movie_ratings
    .groupBy("tag_clean")
    .agg(
        countDistinct("movieId").alias("num_movies"),
        _round(avg("movie_avg_rating"), 2).alias("mean_of_movie_avgs"),
    )
    .orderBy(asc("mean_of_movie_avgs"))
)

print("=" * 80)
print("SUMMARY: WORST TAGS vs GLOBAL AVERAGE")
print(f"Global average rating: {global_avg}")
print("=" * 80)
tag_summary.show(truncate=False)
```

```
=====
SUMMARY: WORST TAGS vs GLOBAL AVERAGE
Global average rating: 3.5
=====
+-----+-----+-----+
|tag_clean|num_movies|mean_of_movie_avgs|
+-----+-----+-----+
|symbolic |1         |0.5                |
|stage    |1         |1.75               |
|tokyo    |1         |2.0                |
|snl      |1         |2.1                |
|jungle   |1         |2.14               |
+-----+-----+-----+
```

Hình 11: So sánh điểm trung bình các tag tệ nhất với điểm trung bình toàn cục.

6.4.5. Nhận xét

- Nếu **mean_of_movie_avgs < global average (3.5)**: Phim mang tag đó thực sự có xu hướng bị đánh giá thấp hơn mặt bằng chung. Tag phản ánh đúng một đặc điểm tiêu cực.
- Nếu **mean_of_movie_avgs $\approx$ hoặc > global average**: Điểm thấp ở cấp tag chỉ do một số ít đánh giá cá biệt, không đại diện cho chất lượng thực sự của phim. Tag không phải chỉ báo đáng tin cậy.
- **Số lượng phim (num_movies)**: Tag chỉ xuất hiện trên 1–2 phim có độ tin cậy rất thấp — kết quả bị chi phối bởi outlier.

Kết luận: Điểm đánh giá trung bình thấp ở cấp tag không nhất thiết đồng nghĩa với việc phim mang tag đó kém chất lượng. Mối quan hệ phụ thuộc vào kích thước mẫu và tính nhất quán của các đánh giá thấp trên nhiều phim.

7. Kết luận

Qua bài thí nghiệm Lab 01, nhóm đã hoàn thành các mục tiêu:

1. **Triển khai hạ tầng:** Cụm Kafka 3 broker hoạt động ổn định trên Docker với chế độ KRaft.
2. **Xử lý dữ liệu streaming:** Đọc thành công dữ liệu từ Kafka, xử lý lỗi định dạng binary bằng pipeline `CAST → from_json → select`.
3. **Phân tích dữ liệu:** Sử dụng Spark DataFrame API để thực hiện các truy vấn phân tích trên tập MovieLens — từ aggregation đơn giản đến phân tích đa chiều kết hợp nhiều bảng.
4. **Tư duy phản biện:** Không dừng lại ở kết quả bề mặt, mà kiểm chứng xem các tag tệ có thực sự phản ánh chất lượng phim hay chỉ là hiệu ứng thống kê do mẫu nhỏ.